JavaScript
Complete Notes

Fundamentals to ES6+ • Functions • Arrays & Objects • Async JS • DOM

Table of Contents

- 1. Introduction to JavaScript
- 2. Variables (var, let, const)
- 3. Data Types
- 4. Operators
- 5. Control Flow (if/else, switch)
- 6. Loops
- 7. Functions
- 8. Arrays
- 9. Objects
- 10. Strings
- 11. ES6+ Features
- 12. DOM Manipulation
- 13. Events
- 14. Asynchronous JavaScript (Callbacks, Promises, Async/Await)
- 15. Error Handling
- 16. Common Array & Object Methods Cheat Sheet

1. Introduction to JavaScript

JavaScript (JS) is a lightweight, interpreted (or JIT-compiled) programming language used to make web pages interactive. It runs in the browser and, via Node.js, on servers too.

- High-level, dynamically typed language.
- Single-threaded with an event loop for handling asynchronous tasks.
- Runs in browsers (client-side) and on servers (Node.js).
- Can be embedded directly in HTML using the <script> tag.

```
<script>
console.log("Hello, World!");
</script>
```

---

2. Variables

JavaScript has three ways to declare variables:

- **var** – function-scoped, can be redeclared and updated (avoid in modern code).
- **let** – block-scoped, can be updated but not redeclared in the same scope.
- **const** – block-scoped, cannot be updated or redeclared (value is constant, but objects/arrays it points to can still be mutated).

```
var x = 10; // function-scoped
let y = 20; // block-scoped
const z = 30; // block-scoped, constant

y = 25; // OK
// z = 35; // Error: Assignment to constant variable
```

Note: Prefer **const** by default, and **let** only when the value needs to change. Avoid **var**.

---

3. Data Types

Primitive Types

- **Number** – e.g. 42, 3.14
- **String** – e.g. "hello"
- **Boolean** – true / false
- **Undefined** – a variable declared but not assigned
- **Null** – intentional absence of value
- **Symbol** – unique identifier (ES6)
- **BigInt** – for numbers larger than Number can hold

Reference Type

- **Object** – includes objects, arrays, and functions

```
typeof 42; // "number"
typeof "hi"; // "string"
typeof true; // "boolean"
typeof undefined; // "undefined"
typeof null; // "object" (a well-known JS quirk!)
typeof {}; // "object"
typeof []; // "object"
typeof function(){}; // "function"
```

---

4. Operators

Type	Operators	Example
Arithmetic	+ - * / % **	5 % 2 -> 1
Assignment	= += -= *= /=	x += 5
Comparison	== === != !== > < >= <=	5 === "5" -> false
Logical	&& || !	true && false -> false
Ternary	condition ? a : b	age >= 18 ? "adult" : "minor"
Nullish coalescing	??	null ?? "default" -> "default"

Note: Always prefer **===** and **!==** (strict equality) over **==** / **!=** to avoid unexpected type coercion.

---

5. Control Flow

```
if (score >= 90) {
console.log("Grade: A");
} else if (score >= 75) {
console.log("Grade: B");
} else {
console.log("Grade: C");
}

switch (day) {
case "Mon":
console.log("Start of week");
break;
case "Fri":
console.log("Almost weekend");
break;
default:
console.log("Midweek");
}
```

---

6. Loops

```
// for loop
for (let i = 0; i < 5; i++) {
console.log(i);
}

// while loop
let i = 0;
while (i < 5) {
console.log(i);
i++;
}

// for...of (iterates over values - arrays, strings)
for (const num of [10, 20, 30]) {
console.log(num);
}

// for...in (iterates over keys - objects)
const person = { name: "Sam", age: 25 };
for (const key in person) {
console.log(key, person[key]);
}
```

7. Functions

```
// Function declaration
function add(a, b) {
return a + b;
}

// Function expression
const subtract = function (a, b) {
return a - b;
};

// Arrow function
const multiply = (a, b) => a * b;

// Default parameters
function greet(name = "Guest") {
return `Hello, ${name}!`;
}

// Rest parameters
function sum(...nums) {
return nums.reduce((total, n) => total + n, 0);
}
```

Note: Arrow functions do not have their own **this** — they inherit it from the surrounding scope.

8. Arrays

```
const fruits = ["apple", "banana", "mango"];

fruits.push("orange"); // add to end
fruits.pop(); // remove from end
fruits.unshift("grape"); // add to start
fruits.shift(); // remove from start

fruits.length; // 3
fruits.includes("mango"); // true
fruits.indexOf("banana"); // 1
fruits.slice(0, 2); // ["grape", "banana"] (does not mutate)
fruits.splice(1, 1, "kiwi"); // replaces 1 item at index 1 (mutates)
```

9. Objects

```
const user = {
name: "Alex",
age: 28,
isAdmin: false,
greet() {
return `Hi, I'm ${this.name}`;
},
};

user.name; // dot notation -> "Alex"
user["age"]; // bracket notation -> 28
user.greet(); // "Hi, I'm Alex"

// Object destructuring
const { name, age } = user;

// Spread operator (shallow copy / merge)
const updatedUser = { ...user, age: 29 };

// Object.keys / values / entries
Object.keys(user); // ["name", "age", "isAdmin", "greet"]
Object.values(user);
Object.entries(user);
```

10. Strings

```
const str = "Hello World";

str.length; // 11
str.toUpperCase(); // "HELLO WORLD"
str.toLowerCase(); // "hello world"
str.includes("World"); // true
str.slice(0, 5); // "Hello"
```

```
str.split(" "); // ["Hello", "World"]
str.trim(); // removes leading/trailing whitespace
str.replace("World", "JS"); // "Hello JS"

// Template literals
const name = "Sam";
const msg = `Hello, ${name}! Today is a ${1 + 1 === 2 ? "good" : "bad"} day.`;
```

11. ES6+ Features

- **let / const** — block-scoped variable declarations
- **Arrow functions** — concise syntax, lexical **this**
- **Template literals** — string interpolation with backticks
- **Destructuring** — unpack values from arrays/objects
- **Spread / Rest (...)** — expand or collect values
- **Default parameters** — fallback values for function args
- **Classes** — syntactic sugar over prototypal inheritance
- **Modules (import/export)** — split code across files
- **Promises & async/await** — cleaner asynchronous code
- **Optional chaining (?.)** and **nullish coalescing (??)**

```
class Animal {
constructor(name) {
this.name = name;
}
speak() {
return `${this.name} makes a sound.`;
}
}

class Dog extends Animal {
speak() {
return `${this.name} barks.`;
}
}

const d = new Dog("Rex");
console.log(d.speak()); // "Rex barks."

// Optional chaining & nullish coalescing
const city = user?.address?.city ?? "Unknown";
```

12. DOM Manipulation

```
// Selecting elements
document.getElementById("title");
```

```
document.querySelector(".card");
document.querySelectorAll("li");

// Changing content
const el = document.querySelector("#title");
el.textContent = "New Title";
el.innerHTML = "<b>Bold Title</b>";

// Changing styles / classes
el.style.color = "blue";
el.classList.add("active");
el.classList.remove("active");
el.classList.toggle("active");

// Creating & appending elements
const li = document.createElement("li");
li.textContent = "New item";
document.querySelector("ul").appendChild(li);
```

---

13. Events

```
const btn = document.querySelector("button");

btn.addEventListener("click", (e) => {
console.log("Button clicked!", e.target);
});

// Common events: click, submit, input, change,
// keydown, keyup, mouseover, mouseout, load
```

---

14. Asynchronous JavaScript

Callbacks

```
function fetchData(callback) {
setTimeout(() => {
callback("Data loaded");
}, 1000);
}

fetchData((result) => console.log(result));
```

Promises

```
const promise = new Promise((resolve, reject) => {
const success = true;
success ? resolve("It worked!") : reject("It failed!");
});
```

```
promise
.then((result) => console.log(result))
.catch((error) => console.log(error))
.finally(() => console.log("Done"));
```

Async / Await

```
async function getUser() {
try {
const response = await fetch("https://api.example.com/user");
const data = await response.json();
console.log(data);
} catch (error) {
console.log("Error:", error);
}
}
```

Note: async/await is just syntactic sugar over Promises — it makes asynchronous code read like synchronous code.

---

15. Error Handling

```
try {
const result = riskyOperation();
} catch (error) {
console.log("Something went wrong:", error.message);
} finally {
console.log("This always runs");
}

// Throwing custom errors
function divide(a, b) {
if (b === 0) throw new Error("Cannot divide by zero");
return a / b;
}
```

---

16. Common Array & Object Methods Cheat Sheet

Method	Description
map()	Transforms each item, returns a new array
filter()	Returns items matching a condition
reduce()	Reduces array to a single value
forEach()	Runs a function on each item (no return)
find()	Returns the first matching item

some() / every()	Checks if some/all items match a condition
sort()	Sorts the array (mutates original)
Object.assign()	Copies properties into a target object
JSON.stringify()	Converts an object/array to a JSON string
JSON.parse()	Converts a JSON string to an object/array

```
const nums = [1, 2, 3, 4, 5];

nums.map(n => n * 2); // [2, 4, 6, 8, 10]
nums.filter(n => n % 2 === 0); // [2, 4]
nums.reduce((sum, n) => sum + n, 0); // 15
nums.find(n => n > 3); // 4
nums.some(n => n > 4); // true
nums.every(n => n > 0); // true
```